ResRec — คู่มือเข้าใจโปรเจกต์ทั้งระบบ สำหรับเตรียมตอบคำถามอาจารย์

Restaurant Recommendation From Reviews — NLP Group Project

ทีม Deesampan Vongsurit (6510766), Wattanan Jiratriluk (6520214)

โค้ดที่อ้างอิง commit `70a8859` บน main (โครงสร้างใหม่ + คะแนนร้านนับเฉพาะความเห็นต่ออาหาร): `src/run_pipeline.py` + `src/train_model.py` + หน้าเว็บไฟล์เดียว `output_project/web/index.html` — เลขบรรทัด (ไฟล์:บรรทัด) ทุกจุดตรวจกับเวอร์ชันนี้แล้วด้วยสคริปต์

วันที่ 18 ตุลาคม 2026 (ฉบับที่ 9 — คะแนนร้านนับเฉพาะความเห็นต่ออาหาร)

> **⚠ เอกสารนี้ AI เป็นคนร่าง — ทีมต้องตรวจและเล่าด้วยคำพูดของตัวเอง**
> เอกสารนี้ Claude (AI) เขียนขึ้นจากการอ่านโค้ดจริงและรันทดสอบจริงในเครื่องนี้ ตัวเลขทุกตัวมาจากไฟล์ผลลัพธ์ใน repo หรือจากการรันจริง แต่ **นี่เป็นงานในรายวิชา** ทีมต้องอ่านทวน ตรวจว่าถูก และเรียบเรียงคำตอบเป็นภาษาของตัวเอง อย่านำข้อความนี้ไปส่งเป็นงานเขียนของตัวเองโดยไม่บอกอาจารย์

สารบัญ

1. **Workflow ทั้งระบบ — แต่ละขั้นใช้เครื่องมืออะไร** (ตอบคำถามหลัก)
2. โฟลเดอร์และไฟล์ — อะไรอยู่ที่ไหน ไฟล์เก่าหายไปไหน
3. โปรเจกต์นี้ AI ทำ หรือเอามาจาก open source?
4. การกำจัด "noise" — เกณฑ์ที่รีวิวจะถูกตัดทิ้ง
5. หัวใจของ NLP: หาเมนู → อ่านความรู้สึกรายอนุประโยค
6. วิธีพิสูจน์ว่าเว็บใช้ NLP จริง (ทดสอบแล้ว พร้อมผลก่อน/หลัง)
7. คำถามที่อาจารย์น่าจะถาม พร้อมแนวคำตอบ
8. เรื่องที่ยังค้างอยู่

1. Workflow ทั้งระบบ — แต่ละขั้นใช้เครื่องมืออะไร

> **สิ่งที่ทีมเข้าใจตอนนี้ (คำถามที่ถามมา)**
>
> *"data from zomato (messy data) and clean it to be as top3_restaurants_reviews.csv then => using nltk to delete noise ... => then use scikit-learn (tf-idf, logistic regression) for giving score to each comment, and then plot to website"*

คำตอบ: โครงใหญ่ถูกแล้ว — ข้อมูลรกจาก Zomato → ทำความสะอาด → โมเดล TF-IDF + Logistic Regression ของ scikit-learn → แสดงบนเว็บ แต่มี **3 จุดที่ต้องแก้**:

#	ที่เข้าใจ	ที่ถูกต้อง
1	**Zomato → ทำความสะอาด → `top3_restaurants_reviews.csv` ไฟล์เดียว**	**ข้อมูล Zomato ใช้ทำ 2 อย่างแยกกัน:** (ก) **ข้อมูลเทรน** 134,612 รีวิวที่ทำความสะอาดแล้ว (`reviews_clean.csv`) ใช้สอนโมเดล และ (ข) **รีวิวของเว็บ** 1,400 รีวิวของ 3 ร้าน (`site_reviews.csv` — ไฟล์นี้คือ `top3_restaurants_reviews.csv` เดิม เปลี่ยนชื่อแล้ว) โมเดลไม่เคยเห็นรีวิวของ 3 ร้านนี้ตอนเทรน
2	**ใช้ NLTK ลบ noise**	**ไม่ได้ใช้ NLTK แล้ว** (เคยใช้ในรุ่นก่อน แต่ถูกเอาออก) การลบ noise ใช้: **ฝั่งเว็บ** — `re` (regular expression) + `unicodedata` ซึ่งเป็นโมดูลมาตรฐานของ Python กับกฎ N1–N4 / **ฝั่งข้อมูลเทรน** — `hashlib` (หาข้อความซ้ำ), `ftfy` (ซ่อมตัวอักษรเพี้ยน), `re` (ลบลิงก์ อีเมล เบอร์โทร), `langdetect` (เก็บแต่ภาษาอังกฤษ)
3	**ให้คะแนนแต่ละคอมเมนต์ แล้ว plot ขึ้นเว็บ**	**ให้คะแนนแต่ละ "อนุประโยค" ไม่ใช่ทั้งคอมเมนต์** และต้อง **หาชื่อเมนูก่อน** (ใช้ RapidFuzz ช่วยจับคำสะกดผิด) เพราะคอมเมนต์เดียวชมเมนูหนึ่งแต่ติอีกเมนูได้ (คะแนนทั้งคอมเมนต์ไม่ได้แสดงบนเว็บ ใช้แค่เซ็กความแม่นกับดาวในรายงาน) ส่วน "plot" คือ **แถบสีที่สร้างด้วย HTML/CSS ใน `index.html`** ไม่มีไลบรารีกราฟ

ภาพรวม: 2 สคริปต์

A. เทรน — `src/train_model.py` (รันครั้งเดียว ใช้เวลา ~10 นาที)

`data_raw/zomato.csv` → A1 ทำความสะอาด → `reviews_clean.csv` → A3 เทรน + ทดสอบ → `sentiment_tfidf_lr.pkl`

`data_raw/zomato.csv` → A2 เลือก 3 ร้าน → `site_reviews.csv + site_restaurants.json`

B. เว็บ — `src/run_pipeline.py` (รันเมื่อไหร่ก็ได้ ไม่กี่วินาที; `start.bat` รันให้)

`site_reviews.csv + .json + .pkl` → B1 โหลด → B2 ลบ noise → B3 หาเมนู → B4 อ่านความรู้สึกรายอนุประโยค → B5 นับ → B6 ส่งออก → `web/data.js` → `web/index.html`

A. เทรน — `src/train_model.py`

ขั้น	เข้า	ออก	เครื่องมือ	โค้ด
A1 ทำความสะอาดข้อมูลทั้งชุด	`zomato.csv` 51,717 แถว = 1,317,782 รีวิว (รก ซ้ำเยอะ)	`data/reviews_clean.csv` **134,612** รีวิว + ป้าย บวก/กลาง/ลบ และ `results/cleaning_funnel.csv`	`ast` อ่าน list รีวิว, `hashlib` หาซ้ำ, `ftfy` + `html` + `re` ซ่อมข้อความ, `langdetect` เก็บภาษาอังกฤษ, ดาว → ป้าย	`src/train_model.py:101-140` `src/train_model.py:82-89` `src/train_model.py:92-98` `src/train_model.py:75-79`

A2 เลือก 3 ร้าน	`zomato.csv`	`data/site_reviews.csv` **1,400** รีวิว (ร้าน, ดาว, ข้อความ) + `data/site_restaurants.json` (เมนู)	`hashlib` นับรีวิวไม่ซ้ำ, `ast` อ่านเมนู, `csv` / `json`	`src/train_model.py:146-183`
A3 เทรน + ทดสอบ	`reviews_clean.csv` **ลบ 3 ร้านบนเว็บออก** = 133,389 รีวิว	`models/sentiment_tfidf_lr.pkl`, `results/model_metrics.json`, `results/confusion_matrix.png`	**scikit-learn**: `TfidfVectorizer` + `LogisticRegression`, `train_test_split`, ตัววัดผล; **vaderSentiment** แค่ตัวเปรียบเทียบ; **matplotlib** วาดรูป; `pickle` บันทึกโมเดล	`src/train_model.py:222-257` `src/train_model.py:207-212` `src/train_model.py:226-228` `src/train_model.py:215-219`

รันครบทั้ง 3 ขั้นแล้วเมื่อ 17 ต.ค. 2026 กับไฟล์ Kaggle ตัวจริง (ใช้เวลา 9 นาที 54 วินาที) — `reviews_clean.csv`, `cleaning_funnel.csv`, `site_reviews.csv`, `site_restaurants.json`, `model_metrics.json` ออกมา**เหมือนไฟล์ใน repo ทุกไบต์**; โมเดลมีคำศัพท์และค่าน้ำหนัก**เหมือนเดิมทุกตัว** (ไฟล์ `.pkl` ต่างกันแค่ตัวเลขภายในหนึ่งตัวที่ scikit-learn จดไว้ คือตำแหน่งในหน่วยความจำของรายการ stopword ซึ่งเปลี่ยนทุกครั้งที่รัน) และ `data.js` ไม่เปลี่ยน

train_model.py เลือก 3 ร้านด้วยไหม? และเทรนกับไฟล์ zomato.csv ทั้งไฟล์หรือเปล่า?

เลือก 3 ร้านด้วย — ขั้น A2 `src/train_model.py:146` นับ**รีวิวไม่ซ้ำ**ของแต่ละร้านใน `zomato.csv` แล้วเอา 3 ร้านที่**มีรีวิวมากที่สุด** (ไม่ใช่คะแนนสูงสุด): Biriyani Nights 524, Night Food Joint 446, BOX8- Desi Meals 430 รีวิว เขียนรีวิวดิบลง `site_reviews.csv` และเมนูลง `site_restaurants.json` รีวิวของเว็บจึงเป็นรีวิว**ดิบ** แล้ว `run_pipeline.py` ทำความสะอาดเองด้วยกฎ N1–N4

ไม่ได้เทรนกับไฟล์ดิบ — เทรนกับรีวิวที่ทำความสะอาดแล้ว และใช้แค่บางส่วน:

ขั้น	จำนวนรีวิว	โค้ด
`zomato.csv` ไฟล์ดิบ	1,317,782	—
A1 ทำความสะอาด: ลบรีวิวซ้ำ, ลบสแปม, ซ่อมข้อความ, เก็บแค่ภาษาอังกฤษ, ติดป้ายจากดาว	134,612	`src/train_model.py:101`
A3 ลบรีวิวของ 3 ร้านบนเว็บออก โมเดลจะได้ไม่เคยเห็นรีวิวที่จะใช้บนเว็บ	133,389	`src/train_model.py:194`
แบ่ง: **ใช้เรียน**	**90,704**	`src/train_model.py:226-227`
แบ่ง: validation (เก็บไว้ ไม่ได้ใช้เรียน)	16,007	
แบ่ง: ทดสอบ (ได้ผลถูก 82.2%)	26,678	

สรุป: โมเดลเรียนจาก **90,704 รีวิวที่ทำความสะอาดแล้ว** ประมาณ 7% ของข้อมูลดิบ ส่วนต่างส่วนใหญ่คือรีวิวซ้ำ (89% ของไฟล์ดิบซ้ำกัน) ข้อความที่โมเดลอ่านผ่านการ normalise แล้ว (ตัวพิมพ์เล็ก, "soooo" → "soo", "wasn't" → "was not") และป้ายมาจากดาวของรีวิวเอง: 4–5 ดาว = บวก, 3–3.5 = กลาง, 1–2.5 = ลบ `src/train_model.py:75`

site_reviews ใช้เป็นข้อมูล train หรือ test? แล้วผลลัพธ์สุดท้ายอยู่ที่ไหน?

ไม่ใช่ทั้ง train และ test — ก่อนแบ่งข้อมูล `train_model.py` ลบรีวิวทุกรีวิวของ 3 ร้านบนเว็บออกก่อน `src/train_model.py:194` `train_test_split` จึงเห็นแค่ 133,389 รีวิวของร้านอื่น `src/train_model.py:226-227` :

กลุ่ม	จำนวนรีวิว	ใช้ทำอะไร
เรียน	90,704	โมเดลเรียนค่าน้ำหนักของแต่ละคำจากกลุ่มนี้
validation	16,007	เก็บไว้ ไม่ได้ใช้เรียน
ทดสอบ	26,678	ตรวจโมเดลกับรีวิวที่ไม่เคยเรียน → ได้ **82.2%**
3 ร้านบนเว็บ	—	ถูกลบออกก่อนแบ่ง โมเดลไม่เคยเห็นตอนเทรน

train_test_split ทำงานยังไง: สลับลำดับรีวิวแบบสุ่มด้วย seed คงที่ (42) ทุกครั้งที่รันจึงได้ผลเหมือนเดิม แล้วตัดแบ่งเป็นกลุ่มตามตาราง โมเดลเรียนจากกลุ่ม "เรียน" เท่านั้น กลุ่ม "ทดสอบ" จึงให้คะแนนที่ยุติธรรม เพราะโมเดลไม่เคยเห็นรีวิวเหล่านั้น

โมเดล "จำ" ได้ยังไง: เมื่อเทรนเสร็จ สิ่งที่เรียนได้ (รายการคำของ TF-IDF + ค่าน้ำหนักของทุกคำจาก Logistic Regression) ถูกบันทึกลง `output_project/models/sentiment_tfidf_lr.pkl` `src/train_model.py:257` ต่อมา `run_pipeline.py` โหลดไฟล์นี้ `src/run_pipeline.py:210` แล้วใช้ให้คะแนนรีวิวของเว็บ การให้คะแนนไม่ได้เปลี่ยนโมเดลเลย — เหมือนนักเรียน: กลุ่ม "เรียน" คือตำรา, กลุ่ม "ทดสอบ" คือข้อสอบ, ส่วน `site_reviews` คืองานจริงหลังเรียนจบ ที่ไม่เคยเห็นทั้งในตำราและข้อสอบ (การเช็กกับดาว 90.1% บนรีวิวของเว็บจึงเป็นการทดสอบรอบที่สองที่ยุติธรรม)

`site_reviews.csv` คือข้อมูลเข้า ไม่ใช่ผลลัพธ์ — มีแค่รีวิวดิบ (ชื่อร้าน, ดาว, ข้อความ) ไม่มีคะแนน และไม่มีอะไรเขียนคะแนนกลับลงไป **ผลลัพธ์สุดท้ายคือ `output_project/web/data.js`** :

```
site_reviews.csv ──► run_pipeline.py (+ โมเดล .pkl) ──► data.js ──► index.html
  ข้อมูลดิบ            ล้าง, หาเมนู, ให้คะแนนอนุประโยค, นับ     ผลลัพธ์      หน้าเว็บแสดง
```

ให้คะแนนแต่ละอนุประโยค (−1 ถึง +1 → ชอบ / ไม่ชอบ / ไม่เลือกข้าง) `src/run_pipeline.py:229` `src/run_pipeline.py:248` → นับต่อเมนูและต่อร้าน `src/run_pipeline.py:286` → เขียนผลลง `data.js` `src/run_pipeline.py:346` ใน `data.js` มีว่าแต่ละรีวิวพูดถึงเมนูไหนและชอบหรือไม่ชอบ, ยอดของแต่ละเมนู ("8 จาก 11 ชอบ") และ % ที่ชอบอาหารของแต่ละร้าน ส่วนตัวเลข −1 ถึง +1 ไม่ได้เก็บไว้ เก็บแค่ป้ายที่ได้

B. เว็บ — `src/run_pipeline.py`

ขั้น	เข้า	ออก	เครื่องมือ	โค้ด
B1 โหลด	`site_reviews.csv`, `site_restaurants.json`	รีวิว 1,400 รีวิว + เมนูของ 3 ร้าน	`csv`, `json`	`src/run_pipeline.py:42-49`
B2 ซ่อมข้อความ + ลบ noise	1,400 รีวิว	**1,321** รีวิว (79 รีวิวถูกตัด พร้อมเหตุผล)	**`re`** + **`unicodedata`** — กฎ N1 ว่าง, N2 ซ้ำ, N3 สั้นกว่า 2 คำ, N4 ไม่ใช่อังกฤษ	`src/run_pipeline.py:59-69` `src/run_pipeline.py:90-93`
B3 หาเมนู	รีวิว + เมนูของร้าน	ตำแหน่งชื่อเมนูในข้อความ (915 ครั้ง ใน 416 รีวิว)	`re` ตัดคำ + **RapidFuzz** `fuzz.ratio` จับคำสะกดผิด	`src/run_pipeline.py:135-147` `src/run_pipeline.py:155-176` `src/run_pipeline.py:152`

B4 อ่านความรู้สึก	รีวิว + ตำแหน่งเมนู + โมเดล	แต่ละ**อนุประโยค**ได้ป้ายชอบ/ไม่ชอบ/ไม่เลือกข้าง → แต่ละเมนูได้ป้ายของอนุประโยคที่มันอยู่	`re` ตัดอนุประโยค + ซ่อนคำ; **scikit-learn** (โมเดลที่เทรนไว้: `transform` + `predict_proba`); `pickle` โหลดโมเดล; เกณฑ์ ±0.4	`src/run_pipeline.py:229-245` `src/run_pipeline.py:199-202` `src/run_pipeline.py:220` `src/run_pipeline.py:223` `src/run_pipeline.py:34` `src/run_pipeline.py:248-264`
B5 นับ	ป้ายของแต่ละเมนูในแต่ละรีวิว	ต่อเมนู: กี่คนชอบ / ไม่ชอบ + ป้าย "Customers love it" ฯลฯ (122 เมนู, จัดอันดับได้ 34)	Python ล้วน	`src/run_pipeline.py:286-304`
B6 ส่งออก	ผลทั้งหมด	`web/data.js` (202 KB)	`json`	`src/run_pipeline.py:325-347`
B7 แสดงผล	`data.js`	หน้าเว็บ: การ์ดร้าน, รายการเมนูพร้อม**แถบสีชอบ/ไม่ชอบ**	HTML + CSS + JavaScript ล้วน — แถบคือ `<span>` ที่ความกว้างตามจำนวนคน **ไม่มีไลบรารีกราฟ**	`output_project/web/index.html:303-306` `output_project/web/index.html:121-122`

สรุปเครื่องมือ

เครื่องมือ	ใช้ทำอะไร	ชั้น
`re`, `unicodedata` (มากับ Python)	ซ่อมข้อความ, กฎลบ noise, ตัดคำ, ตัดอนุประโยค, ซ่อนคำ	A1, B2, B3, B4
`hashlib` (มากับ Python)	หาข้อความซ้ำ (เทียบลายนิ้วมือ MD5)	A1, A2
ftfy, **langdetect**	ซ่อมตัวอักษรเพี้ยน, เก็บเฉพาะภาษาอังกฤษ	A1
scikit-learn	TF-IDF + Logistic Regression = โมเดลความรู้สึก, แบ่งข้อมูล, วัดผล	A3, B4
RapidFuzz	จับชื่อเมนูที่สะกดผิดเล็กน้อย	B3
vaderSentiment	ตัววัดความรู้สึกสำเร็จรูป — **ใช้แค่เปรียบเทียบ** ไม่ได้ใช้บนเว็บ	A3
matplotlib	วาด confusion matrix	A3
ไม่ได้ใช้: NLTK, spaCy, pandas, Flask, HuggingFace / BERT, ไลบรารีกราฟ `CLAUDE.md:140-142` — เวอร์ชันทุกตัวอยู่ใน `requirements.txt` `requirements.txt:1-10`		

ตัวเลขสำคัญที่ควรจำ

- **ข้อมูล:** "Zomato Bangalore Restaurants" บน Kaggle (Himanshu Poddar) — ไม่ได้ scrape เอง
- **Noise:** 1,317,782 → 134,612 รีวิว (ซ้ำ 89%); เว็บ 1,400 → 1,321 รีวิว
- **โมเดล:** ถูก **82.2%** บน 26,678 รีวิวทดสอบ (VADER สำเร็จรูป 72.7%, เดาคำตอบที่พบบ่อยสุด 60.5%) และบนรีวิวของเว็บที่โมเดลไม่เคยเห็น ตรงกับดาว 90.1%
- **ผลบนเว็บ:** ความเห็นต่ออาหารที่ชอบ Biriyani Nights 52% (106 จาก 202), Night Food Joint 38% (42 จาก 111), BOX8- Desi Meals 81% (196 จาก 241); เมนูที่จัดอันดับได้ 8 / 9 / 17
- **เปิดเว็บ:** ดับเบิลคลิก `output_project/web/index.html` หรือ `start.bat` (สร้างตัวเลขใหม่แล้วเปิดหน้า) `start.bat:5-6` หรือ https://resrec.vercel.app

2. โฟลเดอร์และไฟล์ — อะไรอยู่ที่ไหน

`output_project/` มี 5 โฟลเดอร์ `CLAUDE.md:60-65`

โฟลเดอร์	ไฟล์ข้างใน	สร้างโดย / ใช้ทำอะไร
`data/`	`reviews_clean.csv` (134,612 รีวิว), `site_reviews.csv` (1,400 รีวิว), `site_restaurants.json` (3 ร้าน + เมนู)	A1 / A2 สร้าง; เป็นข้อมูลเข้าของ A3 และของเว็บ
`models/`	`sentiment_tfidf_lr.pkl`	A3 สร้าง; B4 ใช้
`results/`	`cleaning_funnel.csv`, `model_metrics.json`, `confusion_matrix.png`	A1 / A3 สร้าง; ตัวเลขและรูปในรายงาน (รูปย้ายมาจากโฟลเดอร์ `figures/` ที่ลบไปแล้ว)
`web/`	`index.html` (เว็บทั้งเว็บ), `data.js` (ตัวเลข), `img/` (รูปเมนู 6 รูป)	B6 เขียน `data.js`; เปิด `index.html` ด้วยดับเบิลคลิก
`reports/`	`report.md`, `presentation.pptx`, `defense_guide_th.pdf` + `.html`	รายงาน สไลด์ และเอกสารนี้

`data_raw/zomato.csv` (ไฟล์ดิบจาก Kaggle 574,072,999 ไบต์) อยู่**นอก** `output_project/` และ**ไม่อยู่ใน git** `.gitignore:1` ใช้แค่ตอนรัน A1–A2 `src/train_model.py:37` — ตอนนี้โหลดไว้ในเครื่องแล้วที่ `C:\nlp_project\data_raw` ไฟล์ทั้งหมดใน `output_project/` ข้างบน**อยู่ใน repo สาธารณะ** (ดูคำถามที่ 15)

ไฟล์เก่าใน `src/` หายไปไหน

ไฟล์เก่า	ตอนนี้
`load_data`, `preprocess`, `dish_extract`, `sentiment`, `aggregate`, `export_web`, `export_removed`, `common`, `config`, `__init__`	**รวมเข้า `src/run_pipeline.py`** (ขั้น B1–B6)
`build_corpus`, `make_subset`, `train_sentiment`	**รวมเข้า `src/train_model.py`** (ขั้น A1, A2, A3)
`image/` (รูป PNG)	ย้ายเป็นรูป `.webp` ใน `output_project/web/img/`
`app`, `serve`	ลบ — เป็นเซิร์ฟเวอร์ ซึ่งไม่ต้องใช้แล้ว เพราะเว็บเปิดด้วยดับเบิลคลิก
`build_gazetteer`, `vocab_analysis`, `make_deck`, `repair_raw_csv`	ลบ — ไม่มีอะไรเรียกใช้แล้ว
`__pycache__`, `.DS_Store`	ลบ — เป็นไฟล์ที่ Python / macOS สร้างเองอัตโนมัติ ไม่ใช่โค้ด

3. โปรเจกต์นี้ AI ทำ หรือเอามาจาก open source?

> **โค้ดเขียนโดยใช้ AI ช่วย (Claude Code) ภายใต้การกำหนดของทีม ไม่ได้คัดลอกโปรเจกต์ open source มาทั้งก้อน** แต่ใช้**ไลบรารีสำเร็จรูป** (ข้อ 1 "สรุปเครื่องมือ") และ**ชุดข้อมูลสาธารณะ**จาก Kaggle ตัวโมเดลวิเคราะห์ความรู้สึก**เราเทรนเอง**
>
> ประโยคนี้ AI สรุปจากหลักฐานด้านล่าง — ทีมควรปรับให้ตรงกับความจริงว่าใครออกแบบ/ตัดสินใจ/ตรวจส่วนไหน

หลักฐาน	พบอะไร
`report.md` ข้อ 11	ทีม**เปิดเผยไว้แล้ว**ว่าใช้ Claude Code เขียนโค้ดส่วนใหญ่ สร้างเว็บ รันการทดลอง และร่างบางส่วนของรายงานและเอกสารนี้ ส่วนทีมตั้งโจทย์ เลือกข้อมูล ตัดสินใจขอบเขต และตรวจผล `output_project/reports/report.md:449-459`
`CLAUDE.md`	ไฟล์ตั้งค่า "AI agent" ของ repo นี้ `CLAUDE.md:3` ให้ AI เป็น "CTO" ที่วางแผน เขียนโค้ด ออกแบบเว็บ `CLAUDE.md:9`
ประวัติ git	commit แรก `97d6c94` (7 ต.ค. 2026) ใส่ทั้งโปรเจกต์ในครั้งเดียว และทีมตั้งกฎไม่ใส่ชื่อ AI ในข้อความ commit แต่เปิดเผยใน report และเอกสารนี้แทน `CLAUDE.md:191-193` — **ใช้ git ตอบคำถามนี้ไม่ได้** ทั้งทางว่ามีหรือไม่มี
ลักษณะโค้ด	เฉพาะเจาะจงกับข้อมูลชุดนี้ (คอมเมนต์ว่า "Biriyani Nights" เป็นชื่อร้าน ไม่ใช่เมนู `src/run_pipeline.py:161`) ไม่มีหัวไฟล์ license ของโปรเจกต์อื่น → ไม่ใช่โปรเจกต์ open source ที่นำเข้ามา
ข้อมูล	Kaggle, Himanshu Poddar — สิทธิ์ "Data files © Original Authors" (ไม่ใช่ open licence) `output_project/reports/report.md:469-473`

4. การกำจัด "noise"

> **ประโยคที่ควรพูดได้**
> "Noise ขึ้นกับงาน รีวิวที่บ่นเรื่องคนส่งของเป็นรีวิวที่ดี แต่ไม่มีประโยชน์กับการจัดอันดับ**เมนู** เราจึงไม่ลบรีวิวพวกนี้ทิ้ง แต่ไม่เอาไปนับคะแนนเมนู ส่วนของที่เป็นขยะจริง ๆ — รีวิวซ้ำ สแปม ข้อความว่าง ภาษาอื่น — เราลบทิ้งเป็นขั้น ๆ และนับไว้ทุกขั้นว่าทิ้งไปกี่แถว"

ชนิดของ noise และวิธีจัดการ

ชนิด	ตัวอย่าง / ขนาด	จัดการโดย	เครื่องมือ
รีวิวซ้ำ	Zomato ลงร้านเดียวกันหลายหมวด รีวิวจึงซ้ำ (89% ของทั้งหมด)	**ลบ**	`hashlib`
สแปม / ข้อความสำเร็จรูป	ข้อความเดียวกันในหลายร้าน (มีข้อความหนึ่งโผล่ใน 496 ร้าน)	**ลบ** ถ้าเกิน 5 ร้าน	`hashlib`
ว่าง / สั้น / ยาวผิดปกติ	มีแต่ "RATED"; "รีวิว" ยาวสุด 39,312 ตัวอักษร	**ลบ**	Python ล้วน
ไม่ใช่ภาษาอังกฤษ	อักษรฮินดี / กันนาดา	**ลบ**	`langdetect` / สัดส่วนอักษรละติน
"RATED" นำหน้า, ขึ้นบรรทัด	99.9% ของรีวิว	แก้ข้อความ	`re`
ตัวอักษรเพี้ยน (mojibake)	`â€™` แทน `'` (6.0% ของรีวิว)	แก้ข้อความ	`ftfy` / `re`
emoji, ลิงก์, อีเมล, เบอร์โทร	—	แก้ข้อความ	`unicodedata` , `re`
ตัวอักษรยืด, คำย่อ	`gooood` , `didn't`	แก้ข้อความ	`re`
สะกดหลายแบบ	biriyani / briyani / biryani	รวมเป็นคำเดียว	พจนานุกรม + RapidFuzz
ไม่พูดถึงเมนู	"delivery was late" (905 จาก 1,321 รีวิวของเว็บ)	**ไม่ลบ** — นับเป็นรีวิวที่อ่านแล้ว แต่ไม่มีผลกับคะแนนเมนูและคะแนนร้าน	—

A1 — ทำความสะอาดข้อมูลเทรน (ตัวเลขจาก `results/cleaning_funnel.csv`)

ขั้น	เกณฑ์ที่ทำให้รีวิวถูกทิ้ง	โค้ด	เหลือ	ทิ้ง
แตกเป็นรีวิว	51,717 แถว → แต่ละรีวิว; ข้ามแถวที่อ่านไม่ได้ / ข้อความว่าง	`src/train_model.py:47-65`	1,317,782	
ตัดซ้ำ	ร้านเดียวกัน (ชื่อ + ที่อยู่) และข้อความเหมือนกันทุกตัวอักษร (MD5)	`src/train_model.py:109-113`	143,789	1,173,993
สแปม	ข้อความเดียวกันใน**มากกว่า 5 ร้าน**	`src/train_model.py:115-116`	140,055	3,734
ความยาว	หลังซ่อมข้อความ สั้นกว่า **15** หรือยาวกว่า **5,000** ตัวอักษร หรือไม่มีตัวอักษร	`src/train_model.py:118-119`	137,290	2,765
ภาษา	อักษรนอก ASCII **เกิน 25%** หรือ langdetect ไม่ใช่ "en"	`src/train_model.py:92-98`	134,619	2,671
ป้าย	ไม่มีดาว (ทำป้ายไม่ได้)	`src/train_model.py:123`	134,612	7

การตัดซ้ำคือ **99.2%** ของทั้งหมดที่ถูกทิ้ง ข้อความที่ใช้เทรนผ่าน `normalise()` **ตัวเดียวกับที่เว็บใช้** `src/train_model.py:35` `src/run_pipeline.py:193-196` และตั้ง seed ให้ langdetect ได้ผลเดิมทุกครั้ง `src/train_model.py:44`

B2 — ลบ noise ของรีวิวบนเว็บ (1,400 รีวิว)

กฎ	เกณฑ์	โค้ด	ทิ้ง	เหลือ
N1 ว่าง	ซ่อมข้อความแล้วไม่เหลืออะไร (เช่นมีแต่ emoji)	`src/run_pipeline.py:90-93`	6	1,394
N2 ซ้ำ	ร้านเดียวกัน ข้อความเหมือนกัน		3	1,391
N3 สั้นเกิน	น้อยกว่า 2 คำ		70	1,321
N4 ไม่ใช่อังกฤษ	อักษรละตินน้อยกว่า 70% `src/run_pipeline.py:72-74`		0	1,321

ทั้ง 79 รีวิวที่ถูกทิ้งแสดงบนหน้าร้านหัวข้อ "Reviews left out" พร้อมเหตุผล `output_project/web/index.html:367` จาก 1,321 รีวิว มี **416 รีวิวที่พูดถึงเมนู** (747 ความเห็นต่อเมนู)

5. หัวใจของ NLP: หาเมนู → อ่านความรู้สึกรายอนุประโยค

B3 หาเมนู

- พจนานุกรมเมนูของแต่ละร้าน = เมนูจริง + แท็ก dish_liked + เมนูทั่วไป 20 รายการ ไม่นับเครื่องดื่ม (coke, pepsi…) และ combo `src/run_pipeline.py:135-147` `src/run_pipeline.py:125-127`
- รวมคำสะกดหลายแบบเป็นแบบเดียว (biriyani → biryani) `src/run_pipeline.py:112-123`
- ชื่อยาวชนะชื่อสั้น ("chicken biryani" ชนะ "biryani"); คำยาว ≥ 6 ตัวอักษรยอมสะกดผิดเล็กน้อย (RapidFuzz ≥ 88) `src/run_pipeline.py:150-152` `src/run_pipeline.py:35` ; ชื่อร้านไม่นับเป็นเมนู `src/run_pipeline.py:161-165`

B4 อ่านความรู้สึก — ทำไมไม่ให้คะแนนทั้งคอมเมนต์

ตัวอย่าง: *"Kebab was yummy but biryani was bland."* ถ้าให้คะแนนทั้งคอมเมนต์ จะได้คะแนนเดียวสำหรับสองเมนู แต่ระบบตัดก่อนคำว่า but ได้ "Kebab was yummy" (+0.991 → Kebab **ชอบ**) และ "but biryani was bland." (−0.945 → Biryani **ไม่ชอบ**) — ทดสอบแล้ว

1. **ตัดอนุประโยค** หลัง `.` `!` `?` `;` และก่อน but / however / although / though / whereas / except / while `src/run_pipeline.py:199`
2. **ซ่อนคำก่อนให้โมเดลอ่าน**: ชื่อเมนู (ไม่ให้ตัดสินจากชื่อเสียงทั่วบังกาลอร์), คำเรื่องการสั่ง ordered/order/tried/got (โมเดลเรียนว่า "ordered" เป็นคำลบ +4.1 เพราะคนบ่นมักเริ่ม "I ordered…"), but ที่ขึ้นต้น (โมเดลเรียนว่าเป็นสัญญาณรีวิว 3 ดาว) `src/run_pipeline.py:200-202` `src/run_pipeline.py:237-241`
3. **ให้คะแนน** = P(บวก) − P(ลบ) ตั้งแต่ −1 ถึง +1 จากโมเดลที่เทรนไว้ `src/run_pipeline.py:214-226`
4. **เลือกข้างเมื่อมั่นใจ**: ≥ +0.4 = ชอบ, ≤ −0.4 = ไม่ชอบ, ระหว่างนั้น = ไม่เลือกข้าง `src/run_pipeline.py:242-244`
5. **ผูกกับเมนู**: เมนูได้ป้ายของอนุประโยคที่มันอยู่ ถ้าอนุประโยคไม่ได้บอกอะไรเลย ("I ordered biryani.") ให้ยืมอนุประโยคถัดไป ("It was cold.") ถ้าอันนั้นไม่มีเมนูอื่นและพูดถึงอาหาร — เกิด 21 จาก 915 ครั้ง `src/run_pipeline.py:248-264`
6. **นับ**: 1 รีวิว = 1 เสียงต่อเมนู (ครั้งสุดท้ายที่เลือกข้าง) `src/run_pipeline.py:270-272` ; % คิดจากคนที่เลือกข้างเท่านั้น `src/run_pipeline.py:301` ; ต้องมี ≥ 4 ความเห็นถึงจะจัดอันดับ; ชอบ ≥ 80% = "Customers love it", ≤ 40% = "Needs work", 40–60% = "Mixed opinions" `src/run_pipeline.py:31-33` `src/run_pipeline.py:275-283`
7. **% ที่ชอบอาหารของร้าน** = โหวตของทุกเมนูในร้านรวมกัน (ชอบ ÷ ความเห็นต่อเมนูทั้งหมด) นับเฉพาะความเห็นต่ออาหาร คอมเมนต์เรื่องบริการ เช่น "ส่งช้า" ไม่นับ `src/run_pipeline.py:334-335` — โมเดลยังอ่านทั้งรีวิวด้วย แต่ใช้แค่เซ็กกับดาว (90.1%) ไม่ได้แสดงบนเว็บ `src/run_pipeline.py:359`

6. วิธีพิสูจน์ว่าเว็บใช้ NLP จริง

หลักการ: แก้ **ข้อความ** ของรีวิวหนึ่งรีวิว โดยไม่แตะจำนวนดาว แล้วสร้างข้อมูลใหม่ ถ้าตัวเลขของเมนูเปลี่ยน แปลว่าตัวเลขมาจากการอ่านข้อความ ไม่ได้มาจากดาวหรือพิมพ์ใส่ไว้ตายตัว

รีวิวที่ใช้: ร้าน **Biriyani Nights** รีวิว 4.5 ดาว ท่อน *"The dessert that is **gulab jamun is mouth watering and tasty af!**"* → เปลี่ยนเป็น `gulab jamun is stale and tasteless!`

ขั้นตอน

1. เช็กว่า repo สะอาด:

   ```
   git status
   ```

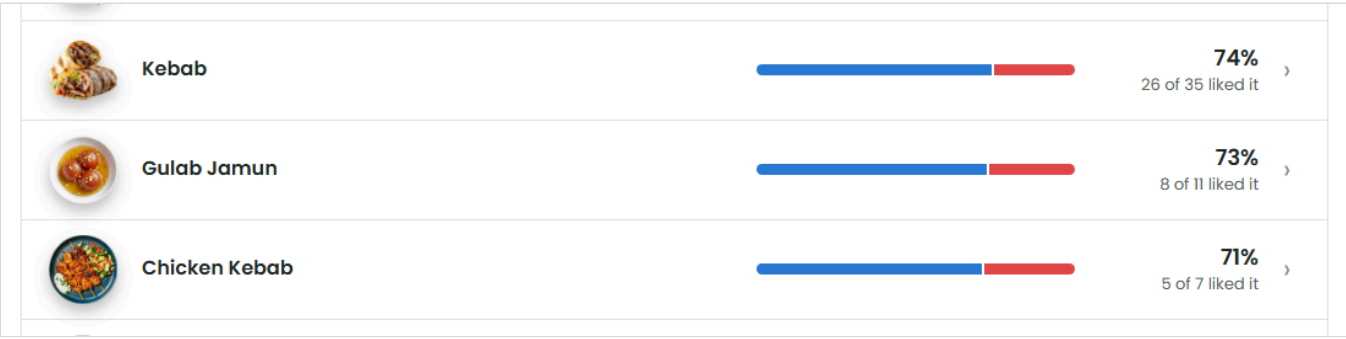
2. เปิด `output_project\data\site_reviews.csv` ด้วย **VS Code หรือ Notepad (ห้ามใช้ Excel** — Excel ตัดช่องที่ยาวเกิน 32,767 ตัวอักษรและเขียนไฟล์ใหม่)
3. ค้นหา `gulab jamun is mouth watering and tasty af!` (มีจุดเดียว) → แก้เป็น `gulab jamun is stale and tasteless!` แล้วบันทึก **ไม่แตะคอลัมน์ rating**
4. ดับเบิลคลิก `start.bat` — รัน `src\run_pipeline.py` (ไม่กี่วินาที) แล้วเปิดหน้าเว็บ (ถ้าหน้าเปิดค้างอยู่ ให้กด F5)
5. คลิก **Biriyani Nights** (หรือเปิด `index.html#biriyani-nights`) แล้วดูแถว **Gulab Jamun**
6. คืนค่าเดิม:

   ```
   git checkout -- output_project/data/site_reviews.csv output_project/web/data.js
   git status
   ```

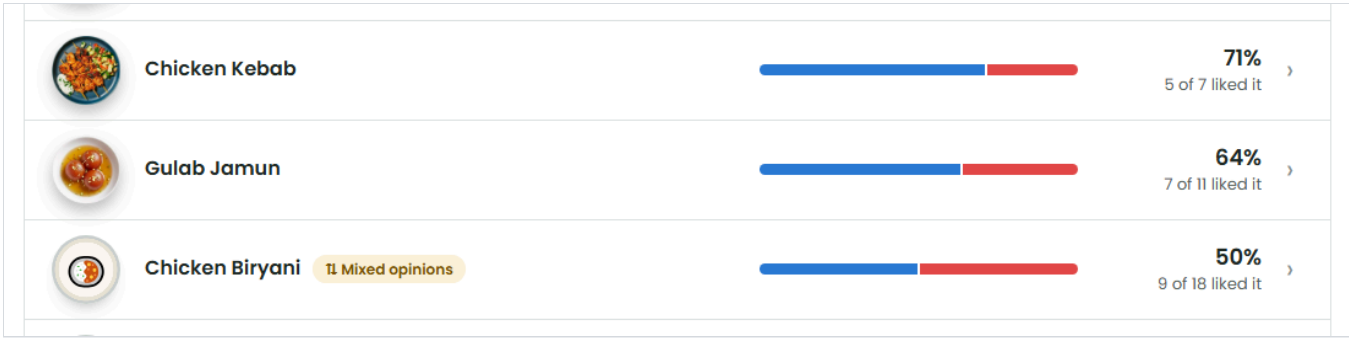
ผลที่ได้จริง (รันแล้ว 17 ต.ค. 2026)

สิ่งที่ดู	ก่อน	หลัง
Gulab Jamun (Biriyani Nights)	**73%** — 8 of 11 liked it	**64%** — 7 of 11 liked it
ลำดับในรายการเมนู	อยู่เหนือ Chicken Kebab	ตกลงไปอยู่ใต้ Chicken Kebab
% ที่ชอบอาหารของร้าน	52% (106 จาก 202)	52% (105 จาก 202 — ลด 1 โหวต)
ดาวของรีวิวนี้	4.5	4.5 (ไม่ได้แตะ)

เปรียบเทียบผล 127 รายการ (3 ร้าน, 122 เมนู, จำนวนรีวิวที่ถูกตัดและที่มีเมนู) **เปลี่ยนแค่รายการเดียว — ดาวไม่เปลี่ยน แปลว่าการเปลี่ยนมาจากข้อความล้วน ๆ** คะแนนของอนุประโยคนั้นเปลี่ยนจาก **+0.984** เป็น **−0.928**



รูปที่ 1: index.html#biriyani-nights **ก่อนแก้** — Gulab Jamun 73%, 8 of 11 liked it



รูปที่ 2: **หลังแก้** — Gulab Jamun 64%, 7 of 11 liked it แถบแดงยาวขึ้น และตกลงไปอยู่ใต้ Chicken Kebab

ทดสอบในสำเนาโค้ดในโฟลเดอร์ชั่วคราว ไม่ได้แก้ repo จริง; ก่อนแก้ `data.js` ที่สร้างได้ตรงกับใน repo ทุกไบต์ (ไม่นับตัวขึ้นบรรทัด); session หลักทดสอบแยกได้ผลเดียวกัน; รูปถ่ายด้วย Chrome แบบ headless จากหน้าเว็บจริง

7. คำถามที่อาจารย์น่าจะถาม พร้อมแนวคำตอบ

แนวคำตอบเหล่านี้ AI ร่าง — ทีมควรซ้อมพูดด้วยคำของตัวเอง ตัวเลขมาจาก `results/model_metrics.json`, `report.md` หรือการรันจริง ตัวอย่างประโยคทุกตัวรันกับโค้ดปัจจุบันแล้ว

1. อธิบาย workflow ทั้งหมดใน 30 วินาที

"เราเอาข้อมูลรีวิว Zomato จาก Kaggle มาทำความสะอาด ได้ 134,612 รีวิวที่มีดาวเป็นป้าย ใช้เทรนโมเดล TF-IDF + Logistic Regression ของ scikit-learn แยกรีวิวของ 3 ร้านไว้ทำเว็บ 1,400 รีวิว ตอนสร้างเว็บ เราลบ noise ด้วย regex, หาชื่อเมนูในแต่ละรีวิว, ตัดรีวิวเป็นอนุประโยค, ให้โมเดลอ่านแต่ละอนุประโยค, ให้เมนูได้ผลของอนุประโยคที่มันอยู่ แล้วนับว่าแต่ละเมนูมีคนชอบกี่คน ผลเขียนลง data.js แล้วหน้าเว็บวาดเป็นแถบสี" (ดูข้อ 1)

2. ใช้ NLTK ลบ noise ใช่ไหม?

ไม่ใช่ (รุ่นก่อนเคยใช้ แต่เอาออกแล้ว) การลบ noise ของเว็บใช้ `re` + `unicodedata` ที่มากับ Python และกฎ 4 ข้อ N1–N4 `src/run_pipeline.py:77-105` ส่วนข้อมูลเทรนใช้ `hashlib` หาซ้ำ, `ftfy` ซ่อมตัวอักษร, `re` ลบลิงก์/อีเมล/เบอร์ และ `langdetect` เก็บภาษาอังกฤษ ใช้ regex เพราะกฎเราง่ายและอธิบายได้ทีละบรรทัด ไม่ต้องพึ่งไลบรารีใหญ่

3. Noise ในข้อมูลนี้คืออะไร?

ใหญ่ที่สุดคือ **รีวิวซ้ำ 89%** (1,317,782 → 143,789) เพราะ Zomato ลงร้านเดียวกันหลายหมวด ตามด้วยสแปมที่ซ้ำเกิน 5 ร้าน (3,734), สั้น/ยาวผิดปกติ (2,765), ไม่ใช่ภาษาอังกฤษ (2,671) และไม่มีดาว (7) — เหลือ 134,612 รีวิวที่ไม่พูดถึงเมนูไม่ได้ลบ แต่ไม่นับคะแนนเมนู ทุกขั้นนับไว้ใน `cleaning_funnel.csv`

4. โมเดลให้คะแนนทั้งคอมเมนต์หรือเปล่า?

ไม่ คะแนนของเมนูมาจาก**อนุประโยค**ที่ซื่อเมนูอยู่ เพราะคอมเมนต์เดียวชมเมนูหนึ่งแต่ติอีกเมนูได้ (ตัวอย่าง Kebab / Biryani ในข้อ 5) คะแนนทั้งคอมเมนต์ไม่ได้แสดงบนเว็บ (ใช้แค่เซ็กกับดาวในรายงาน) และคะแนนร้านก็รวมจากโหวตของเมนู

5. ทำไมไม่ใช้ ABSA (Aspect-Based Sentiment Analysis)?

ABSA ต้องมีข้อมูลที่คนติดป้ายระดับ "เมนู + ความรู้สึก" ซึ่งไม่มี (มีแต่ดาวระดับรีวิว) และซับซ้อนเกินกว่าจะอธิบายได้ในงานวิชานี้ เราใช้วิธีที่ง่ายกว่าแต่ได้ผลระดับเมนูเหมือนกัน ข้อแลกคือถ้าสองเมนูอยู่ในอนุประโยคเดียวกันจะได้ผลเดียวกัน

6. ทำไมเลือก TF-IDF + Logistic Regression ไม่ใช้ BERT?

ง่าย อธิบายได้ เทรนบน CPU ได้ และ**น้ำหนักของคำคือคำอธิบาย**: คำที่ดันไปทางลบมากสุดคือ worst (+12.99), pathetic, not, bad; ทางบวกคือ awesome (+9.94), best, amazing, delicious ผลบน 26,678 รีวิวทดสอบ: **ถูก 82.2%, macro-F1 0.774** ดีกว่า VADER (72.7%, 0.543) และดีกว่าเดาคำตอบที่พบบ่อยสุด (60.5%, 0.251) BERT น่าจะแม่นกว่า แต่หนักกว่า อธิบายยากกว่า และไม่จำเป็นต่อคำถาม "เมนูไหนดี"

7. ป้ายที่ใช้เทรนมาจากไหน? ทำไมต้องตัด 3 ร้านออก?

จากดาวของรีวิว: ≥ 4 = บวก, 3–3.5 = กลาง, < 3 = ลบ `src/train_model.py:75-79` (เป็น "weak label" ไม่มีคนติดป้ายเอง) ตัด 1,223 รีวิวของ 3 ร้านบนเว็บออกก่อนเทรน `src/train_model.py:201` เพื่อไม่ให้โมเดลเคยเห็นรีวิวพวกนี้พร้อมดาวมาก่อน — ไม่งั้นผลบนเว็บจะดูดีเกินจริง

8. ระบบแม่นแค่ไหน? วัดระดับเมนูหรือยัง?

ระดับรีวิว: ถูก 82.2%; คลาส "กลาง" อ่อนสุด (F1 0.582) เพราะรีวิว 3 ดาวกำกวม บนรีวิวของเว็บ (ที่โมเดลไม่เคยเห็น) ตรงกับดาว **90.1%** (1,034 จาก 1,147) `src/run_pipeline.py:307-313` **ระดับเมนู/อนุประโยคยังไม่ได้วัดตรง ๆ** เพราะไม่มีป้ายระดับอนุประโยค — ใช้ดาวเซ็กคร่าว ๆ: "ชอบ" ในรีวิว 1 ดาว หรือ "ไม่ชอบ" ในรีวิว 5 ดาว มี 21 จาก 747 ความเห็น ถ้าทำต่อ จะติดป้ายอนุประโยคด้วยมือสัก 200 อัน

9. ทำไมซ่อนชื่อเมนูกับคำว่า "ordered" และใช้เกณฑ์ 0.4?

โมเดลเรียนจากรีวิวทั้งรีวิว บางคำจึงมีความหมายแฝง ("ordered" ไปทางลบ +4.1, but ขึ้นต้นไปทาง neutral +3.78, ชื่อเมนูมี "ชื่อเสียง" ติดมา) และอนุประโยคสั้นกว่าที่โมเดลเรียน จึงให้เลือกข้างเฉพาะตอนมั่นใจ (±0.4) เซ็กด้วยดาว: ความผิดชัด ๆ ลดจาก 52 เหลือ 21 แลกกับความเห็นที่เลือกข้างลดเหลือ 554 จาก 747 **ข้อควรพูดตรง ๆ:** ค่า 0.4 เลือกด้วยการเซ็กเดียวกันนี้ ตัวเลขจึงเข้าข้างมัน

10. ทำไมไม่เอาคำว่า not / no ออกเหมือน stopword ทั่วไป?

เพราะ "not good" จะกลายเป็น "good" ทดลองแล้ว: เก็บคำปฏิเสธไว้ดีกว่า **+2.1 จุด accuracy** (80.0% → 82.2%) และ +2.5 จุด macro-F1 `src/train_model.py:190-191` `src/train_model.py:236`

11. หาเมนูด้วย NER ไหม? "plot" ขึ้นเว็บใช้อะไร?

ไม่ใช้ NER: โมเดล NER สำเร็จรูป (เช่นของ spaCy) ไม่มีประเภท "อาหาร" และเทรนเองต้องมีข้อมูลติดป้ายที่เราไม่มี จึงใช้พจนานุกรมเมนูของร้าน + RapidFuzz `src/run_pipeline.py:155-176` ส่วนหน้าเว็บไม่มีไลบรารีกราฟ — แถบชอบ/ไม่ชอบคือ `<span>` สองสีที่ความกว้างตามจำนวนคน `output_project/web/index.html:303-306`

12. มีกรณีที่ระบบผิดไหม?

มี ตัวอย่างจริงจากข้อมูล:

- "Loved the colorful box that the sandwich wrap was packed in." (รีวิว 1 ดาว) → Sandwich และ Wrap ถูกนับว่า **ชอบ** ทั้งที่ชมกล่อง
- "We ordered home made dum biriyani and butter chicken." (1 ดาว) → **ชอบ** เพราะโมเดลเรียนว่า "home" (+2.43) และ "dum" (+1.16) เป็นคำบวก
- "but the price is like Paradise biriyani." → Biryani **ชอบ** (การเปรียบเทียบ)
- "The biryani was not bad." ได้ −0.076 → ไม่เลือกข้าง ทั้งที่ค่อนไปทางชม
- "lemonade was not upto mark" ที่ BOX8 ไม่ถูกนับ เพราะเมนูร้านเขียน "Masala Lemonade"; emoji ถูกทิ้ง (TF-IDF อ่านแต่คำ); สองเมนูในอนุประโยคเดียวได้ผลเดียวกัน

13. ทำไมไม่ scrape เอง? เลือก 3 ร้านนี้เพราะอะไร?

Scrape ผิดเงื่อนไขของแพลตฟอร์ม ทำซ้ำไม่ได้ และไม่ใช่งาน NLP ชุดข้อมูลสาธารณะให้คนอื่นตรวจซ้ำได้ ส่วน 3 ร้านนี้คือร้านที่มี**รีวิวไม่ซ้ำมากที่สุด** `src/train_model.py:146-151` ลำดับ % ที่ชอบอาหาร (81% > 52% > 38%) ตรงกับคะแนนบน Zomato (4.1 > 3.2 > 2.9) ทั้งที่เราไม่ได้ใช้คะแนนนั้น

14. ทำไมเมนูต้องมีความเห็นขั้นต่ำ? ความเห็น "กลาง" ไปไหน?

กัน "1 จาก 1 = 100%" ขึ้นเป็นเมนูที่ดีที่สุด ต้องมี ≥ 4 ความเห็นที่เลือกข้าง `src/run_pipeline.py:31` ส่วนรีวิวที่แค่เอ่ยชื่อเมนูโดยไม่เลือกข้าง ไม่ถูกนับใน %

15. ใช้ข้อมูลนี้ได้ถูกต้องตามสิทธิ์ไหม?

ต้องตอบตามจริง: สิทธิ์ "Data files © Original Authors" **ไม่ใช่ open licence** repo บน GitHub **เป็นสาธารณะ** ตั้งแต่ 16 ต.ค. 2026 ตามการตัดสินใจของทีม ไม่มีไฟล์ดิบ `zomato.csv` แต่**มีไฟล์ที่มีข้อความรีวิว** (`reviews_clean.csv` , `site_reviews.csv` , `data.js` ซึ่ง Vercel ก็เสิร์ฟ) **ทีมรู้และยอมรับความเสี่ยงนี้เอง** และเขียนไว้ใน report `output_project/reports/report.md:479-482` ถ้าอาจารย์เห็นว่าไม่ควร ทำ repo กลับเป็น private ได้

16. AI เขียนโค้ดให้หรือเปล่า?

ตอบตามจริง (ดูข้อ 3): ใช้ Claude Code ช่วยเขียนโค้ดส่วนใหญ่ ภายใต้การกำหนดของทีม เปิดเผยไว้ใน `report.md` ข้อ 11 และในเอกสารนี้ ทีมกำหนดโจทย์และขอบเขต (ตัด ABSA, ไม่ scrape, โมเดลเดียว, ตัด 3 ร้านออกจากข้อมูลเทรน) และต้องอธิบายได้ทุกชั้น ไม่มีบริการ AI ทำงานอยู่ในเว็บ — ทุกป้ายความรู้สึกมาจากโมเดลที่เราเทรนเอง (ทีมปรับประโยคนี้ให้ตรงกับความจริง)

17. รันซ้ำแล้วได้ผลเดิมไหม?

ได้ทั้งหมด: วันที่ 17 ต.ค. 2026 โหลดไฟล์ Kaggle ตัวจริงแล้วรัน `train_model.py` ครบทุกขั้น (9 นาที 54 วินาที) — ข้อมูลที่ทำความสะอาด, 3 ร้านที่เลือก และตัวเลขผลทดสอบ **ออกมาเหมือนเดิมทุกไบต์** โมเดลได้ค่าน้ำหนักเหมือนเดิมทุกตัว (ไฟล์ `.pkl` ต่างแค่ตัวเลขภายในหนึ่งตัวที่ scikit-learn จดไว้ ซึ่งเปลี่ยนทุกครั้งที่รัน) และ `run_pipeline.py` สร้าง `data.js` ได้เหมือนเดิม เพราะตั้ง seed ไว้ (แบ่งข้อมูล 42, langdetect 0) `src/train_model.py:226-228`

18. train_model.py เลือก 3 ร้านด้วยใช่ไหม? เลือกยังไง?

ใช่ ขั้น A2 `src/train_model.py:146` เลือกจาก**จำนวนรีวิวไม่ซ้ำมากที่สุด** ไม่ใช่คะแนนสูงสุด เพราะต้องมีรีวิวมากพอให้แต่ละเมนูมีความเห็นพอจะตัดสิน (ร้านส่วนใหญ่มีรีวิวแค่ 3 รีวิว) ได้ Biriyani Nights 524, Night Food Joint 446, BOX8- Desi Meals 430 รีวิว แล้วลำดับ % ที่ชอบอาหารบนเว็บก็ออกมาตรงกับคะแนน Zomato ทั้งที่เราไม่ได้ใช้คะแนนนั้นในการเลือก (ดูกล่องในข้อ 1)

19. โมเดลเทรนกับ zomato.csv ทั้งไฟล์เลยไหม?

ไม่ เทรนกับ**รีวิวที่ทำความสะอาดแล้ว**: 1,317,782 รีวิวดิบ → ทำความสะอาดเหลือ 134,612 `src/train_model.py:101` → ลบ 3 ร้านของเว็บออกเหลือ 133,389 `src/train_model.py:194` → แบ่งเป็น **ใช้เรียน 90,704**, validation 16,007 และทดสอบ 26,678 `src/train_model.py:226-227` โมเดลจึงเรียนจากประมาณ 7% ของข้อมูลดิบ เพราะ 89% ของไฟล์ดิบเป็นรีวิวซ้ำ (ดูกล่องในข้อ 1)

20. site_reviews ใช้เป็นข้อมูล train หรือ test?

ไม่ใช่ทั้งสองอย่าง รีวิวของ 3 ร้านบนเว็บถูกลบออกก่อนแบ่งข้อมูล `src/train_model.py:194` `train_test_split` แบ่งเฉพาะ 133,389 รีวิวของร้านอื่นเป็น เรียน 90,704 / validation 16,007 / ทดสอบ 26,678 `src/train_model.py:226-227` เทรนเสร็จโมเดลถูกบันทึกเป็น `.pkl` `src/train_model.py:257` แล้ว `run_pipeline.py` โหลดมาให้คะแนนรีวิวของเว็บ `src/run_pipeline.py:210` โมเดลจึงไม่เคยเห็นรีวิวของเว็บมาก่อน (ดูกล่องในข้อ 1)

21. site_reviews.csv คือผลลัพธ์หลังให้คะแนนแล้วใช่ไหม?

ไม่ใช่ `site_reviews.csv` คือ**ข้อมูลเข้า** มีแค่รีวิวดิบ ไม่มีคะแนน ผลลัพธ์สุดท้ายคือ **`output_project/web/data.js`** ที่ `run_pipeline.py` เขียน `src/run_pipeline.py:346` หลังให้คะแนนอนุประโยค `src/run_pipeline.py:229` และนับต่อเมนู `src/run_pipeline.py:286` หน้าเว็บอ่านแค่ `data.js` ถ้าแก้ `site_reviews.csv` หน้าเว็บจึงไม่เปลี่ยนจนกว่าจะรัน `run_pipeline.py` ใหม่ (ซึ่งคือสิ่งที่การทดสอบในข้อ 6 แสดงให้เห็น)

8. เรื่องที่ยังค้างอยู่

#	เรื่อง	รายละเอียด / ผลกระทบ
1	ความแม่นระดับอนุประโยคยังไม่ได้วัด	มีแค่การเช็กคร่าวด้วยดาว (21 จาก 747) — คำถามที่ 8, 9
2	Repo สาธารณะและ Vercel มีข้อความรีวิว	สิทธิ์ข้อมูลไม่ใช่ open licence — **ทีมตัดสินใจยอมรับความเสี่ยงนี้เอง** `CLAUDE.md:155-160` (คำถามที่ 15)
3	ไอคอนบางเมนูเป็นกล่องว่างบน Windows 10	emoji รูปแผ่นแป้ง (U+1FAD3) ของ Paratha, Naan, Roti ไม่มีในฟอนต์ของ Windows 10
4	ยังไม่ได้ตรวจด้วยมือ	ยังไม่ได้สุ่มรีวิวที่ถูกทิ้งมาดู และเกณฑ์ 15 ตัวอักษร / 2 คำยังไม่ได้ลองค่าอื่น (ระบุใน Limitations ของ report)

จบเอกสาร — ร่างโดย AI (Claude) จากโค้ด commit 70a8859 และการรันทดสอบจริงวันที่ 17–18 ต.ค. 2026 ทีมต้องตรวจทานก่อนใช้